Roll No.: 50

Name: Sebin S. Jacob

SAINTGITS
LEARN GROW EXCEL

Saintgits College of Engineering, Kottayam (Autonomous)
Kottukulam Hills, Pathamuttom P.O, Kottayam-686 532

SIXTH SEMESTER B.TECH. INTERNAL TEST – I, JANUARY 2026

Course Code : 20CST306
Course Name : Algorithm Analysis and Design
Applicable to: CS & ER

Time : 2 Hrs

Max. Marks : 50.00

Part A (Answer All Questions)

1. Algorithm A uses $10nlogn$ operations, while algorithm B uses n^2 operations. Determine the value n_0 such that A is better than B for $n \geq n_0$. [Mark : 5]

2. Order the following in the increasing order of growth. and group together those functions of same order.

$6nlogn, 2^{100}, loglogn, log^2n, 2^{logn}, 4^n, sqrt(n), n^{0.01}, \frac{1}{n},$

$4n^{3/2}, 3n^{0.5}, 5n, 2nlog^2n, 2^n, nlog^4n, 4^n, n^3, n^2logn, 4^{logn}, sqrt(logn).$ [Mark : 5]

3. (a) Does the complexity of an algorithm differ in different architecture.

(b) Is space complexity important ? State your answer with a sample problem. [Mark : 5]

4. Explain the significance of balanced binary search tree with given example. Key nodes: 50, 45,32,27,22,15,10,5. What is the main drawback of BST. [Mark : 5]

Part B (Answer All Questions)

5. Explain about asymptotic notations? What does the term 'asymptotic' means. Also Draw the growth of a funtion with respect to the number of instructions to be executed [Mark : 10]

Scanned with OKEN Scanner

6. Solve the recurrence relation and find the complexity [Use any method].

(a) $T_n = \begin{cases} 1 & if \quad n = 2 \\ T(\sqrt{n}) + 1 & if \quad n > 2 \end{cases}$

(b) $T_n = 2T(n/2) + n/logn$

(c) $T_n = T(n-1) + 2^n$

[Mark : 10]

7.(a) Explain AVL tree and its different rotations to adjust balancing its height.

[Choose the given BST with keys (i) 30, 20, 10, (ii) 30,10,20, (iii) 10,20,30, and (iv) 10,30,20 ; to explain the rotations.]

(b) Construct AVL Tree; Keys : **40, 20, 10, 25, 30, 22, 50**

[Mark : 10]

Mapping Questions to Course Outcomes

Sections	**Part A**				**Part B**		
Qn No.	Q1	Q2	Q3	Q4	Q5	Q6	Q7
Course Outcomes	CO1	CO1	CO1	CO2	CO1	CO1	CO2
Taxonomy Level	2	2	2	3	2	3	3

***

Scanned with OKEN Scanner

Part A

1. A: $10n\log n$

B: n^2

To show

$f(n) \le c \cdot g(n)$

$10n\log n \le 10n \times n$

$n=1 \Rightarrow 0 \le 10$ ✓

So

$f(n) \le c \cdot g(n)$

$n^2 \le$

$f(n) \le c \cdot g(n)$

$n^2 \le n^2$

$n=1$

$1 \le 1$

$O(n^2)$

$f(n) \ge c \cdot g(n)$

$10n\log n \ge 10n$

$f(n) \le c \cdot g(n)$

$10n\log n \le 10n$

$n=1$ $0 \le 10$ ✓

$O(n\log n)$

$O(n\log n) < O(n^2)$

$\therefore$ A is better.

Scanned with OKEN Scanner

2.

$n^{0.01} < \text{sqrt}(\log n) < \text{sqrt}(n) < 3n^{0.5} < 2^{\log n}$

$< 4^{\log n} <$

$\log\log n < 1/n < n^{0.01} < \text{sqrt}(\log n) < \text{sqrt}(n) < 2^{\log n} < 3n^{0.5} < 4^{\log n}$

$< 3.n^{0.5} < \log^2 n < 6n\log n < n\log^4 n < 2n\log^2 n$

$< 5n < 4n^{3/2} < 2^n < 4^n, n^3 < 2^{100}$

$\log\log n < 1/n < n^{0.01} < \text{sqrt}(\log n) < \text{sqrt}(n) < 2^{\log n} < 4^{\log n}$

$< 3.n^{0.5} < \log^2 n < 6n\log n < n\log^4 n < 2n\log^2 n$

$< 5n < 4n^{3/2} < 2^n < 4^n, n^3 < 2^{100}$

3.

a) No, complexity of algorithm does not depend upon machine or architecture

As

complexity is calculated as no: of statements executed

~~for time~~

complexity is used for performance analysis not performance measurement.

Scanned with OKEN Scanner

b) Yes

Space complexity can affect

As size of program increases

- Both memory required and complexity of the program increase thus making the program more costly.

eg: In a large project

There will be multiple variable and function that can consume large space of memory which may be fixed space

$S(P) =$ fixed + variable

depending upon the input the space can further increase thus requiring more memory

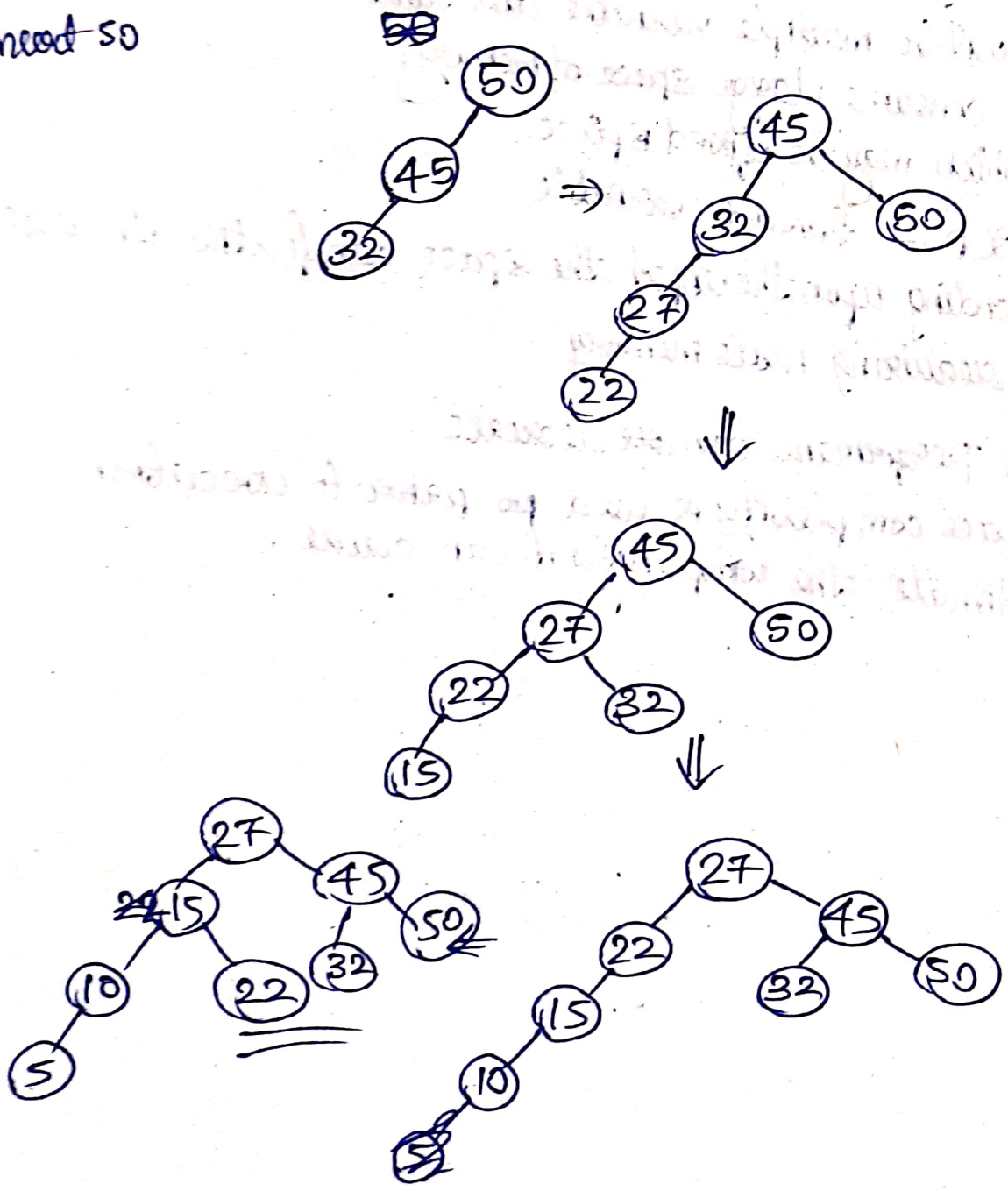
Due this performance can decrease

So, space complexity is used prior to execution to estimate the complexity it can cause.

4. Balanced binary search tree

- The complexity of BST is O(logn)
  worst case it can O(n)
- The elements are in sorted order
- Insertion/ deletion from balanced BST is O(logn)
- faster searching

Insert 50



Part: B

5. Asymptotic notation

There are mainly 3 notation

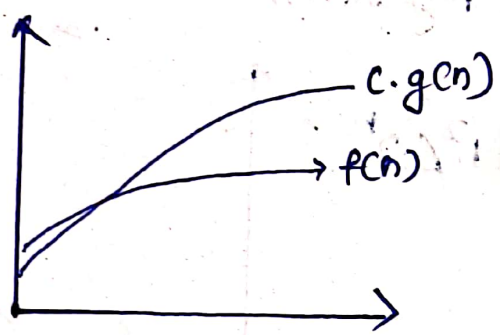
i) Bigoh notation (O)

Bigoh is used to represent worst case ~~scene~~ of function

$f(n) \leq c.g(n)$

iff $n \geq n_0$ & $c \geq 1$

$f(n) = O(g(n))$



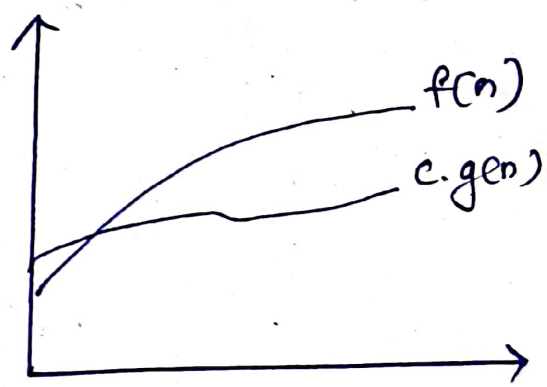
ii) Omega notation (Ω)

Omega is used to represent best case of function

$f(n) \geq c.g(n)$

iff $n \geq n_0$ & $c \geq 1$

$f(n) = \Omega(g(n))$



Scanned with OKEN Scanner

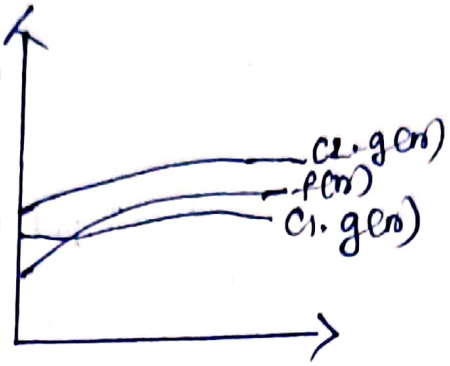
iii) Theta notation (θ)

It is used to represent the average case of function.

$$c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$$

if $n \ge n_0$

$$f(n) = \theta(g(n))$$

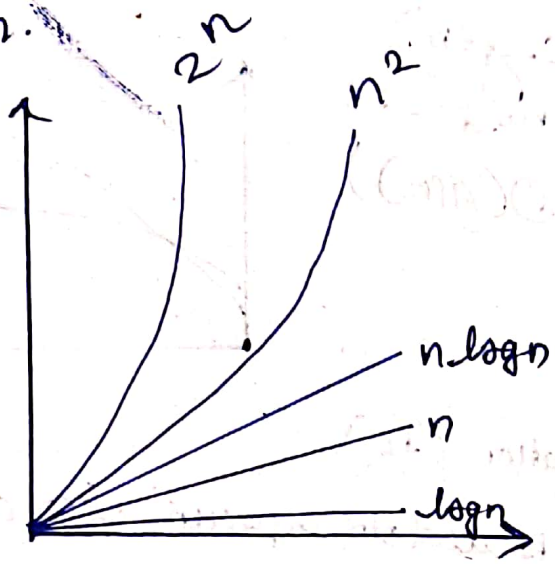


iv) little oh

v) little omega

These asymptotic notations are used to show the optimality of algorithm.

$\log n < n < n\log n < n^2 < 2^n$



Scanned with OKEN Scanner

6. a) $T_n = \begin{cases} 1 & \text{if } n=2 \\ T(\sqrt{n})+1 & \text{if } n>2 \end{cases}$

$$T(n) = T(\sqrt{n}) + 1$$

$$= T(n^{1/4}) + 1 + 1$$

$$= T(n^{1/8}) + 1 + 1 + 1$$

$$\Rightarrow T(n^{1/2^3}) + 3$$

$$\Rightarrow T(n^{1/2^K}) + K$$

$$2^K = n$$

$$\log n = K$$

$$n^{1/2K} = 2$$

$$\cancel{m = 1/2K} \qquad \log_2 n = 1/2K$$

$$\cancel{n^m = 2} \qquad n^{\log_2 n}$$

$$\cancel{\log_n m = 2} \qquad 2K = 1/\log n$$

$$K = \frac{1}{2\log n}$$

$$2 + K = 2 + \frac{1}{2\log n} = \text{constant} \quad O(1)$$

Scanned with OKEN Scanner

b) $T_n = 2T(n/2) + n/\log n$

$a=2 \quad b=2$

$f(n) = n/\log n$

$n^k \log^p n = n/\log n$

$k=1 \quad p=-1$

$a = b^k$

case 2

$p=-1 \quad = O(n^k \log(\log n))$

$= O(n \log(\log n))$

c) $T_n = T(n-1) + 2^n$

$a=1$ ~~b=1~~

$f(n) = 2^n$

$a=1$

$O(n.f(n))$

$= O(n.2^n)$

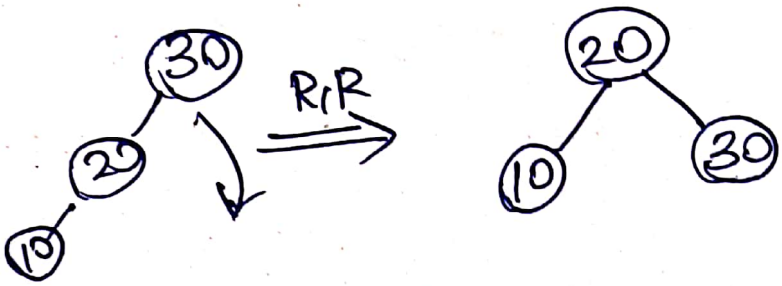
Scanned with OKEN Scanner

AVL Tree : It is self balancing tree where the difference between in height of ~~right~~ left subtree & right subtree should be ~~then~~ or equal to −1

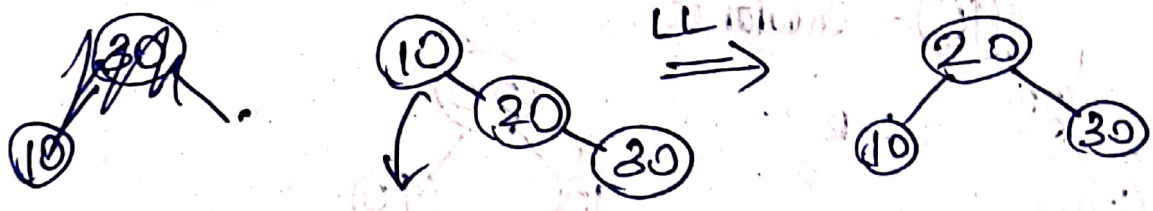
$$d = h(L) - h(R)$$

There are 4 rotation

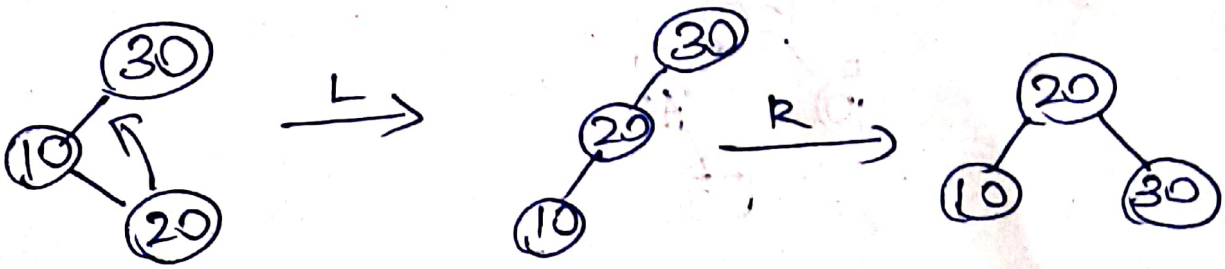
i) Right rotation (RR)



ii) left rotate (LL)

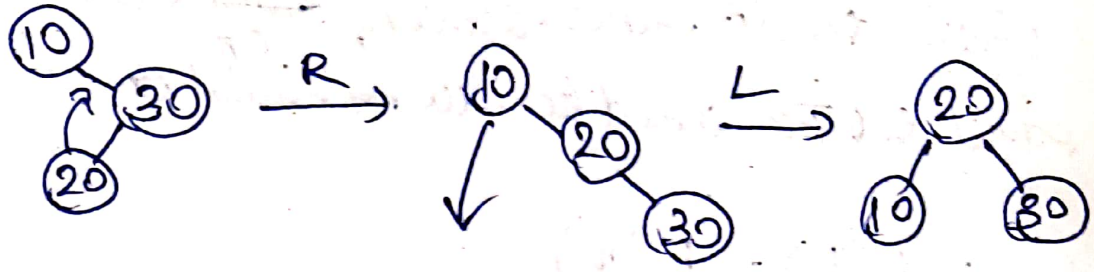


iii) LR (left right rotate)



Scanned with OKEN Scanner

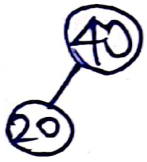
iv) RL (Right left)



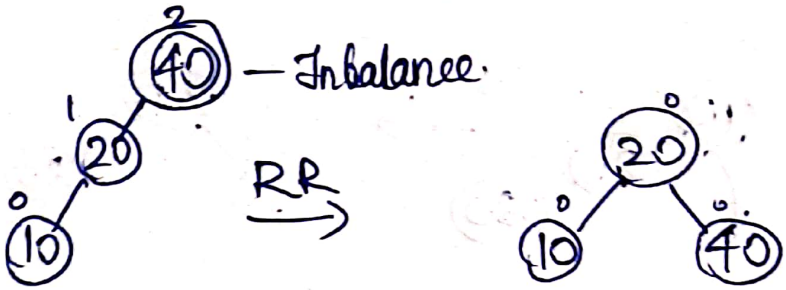
b)

Insert 40

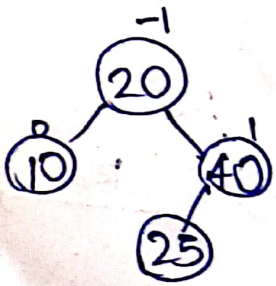
Insert 20



Insert 10

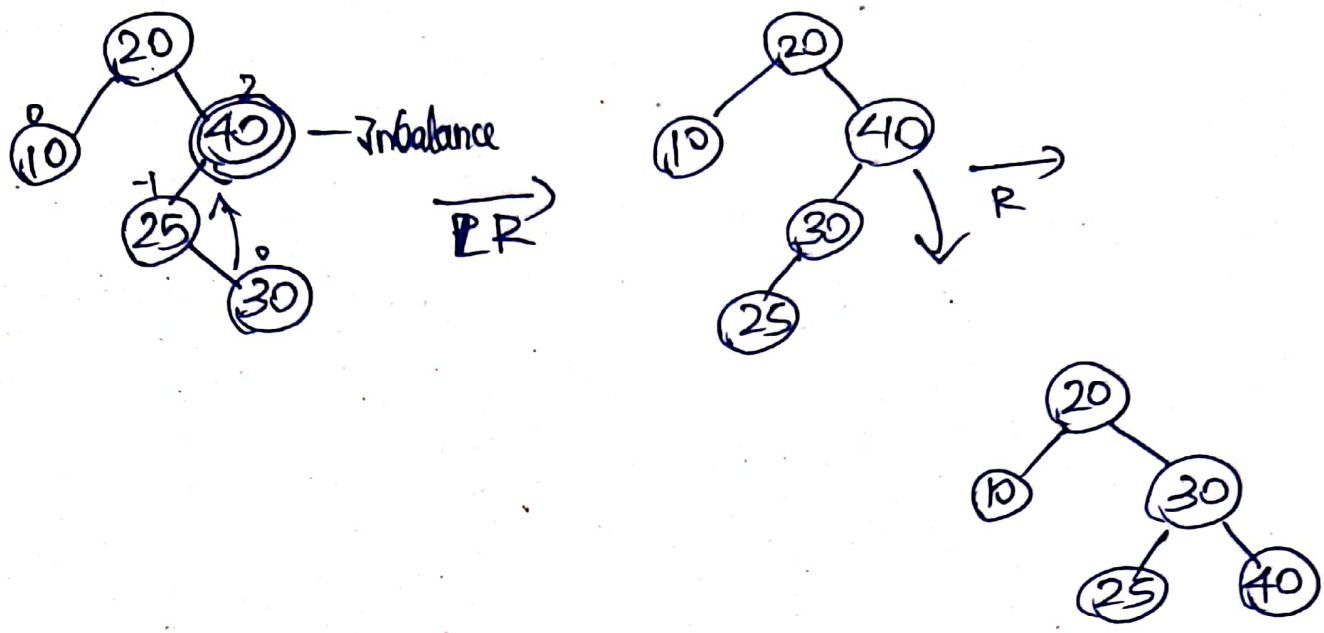


Insert 25

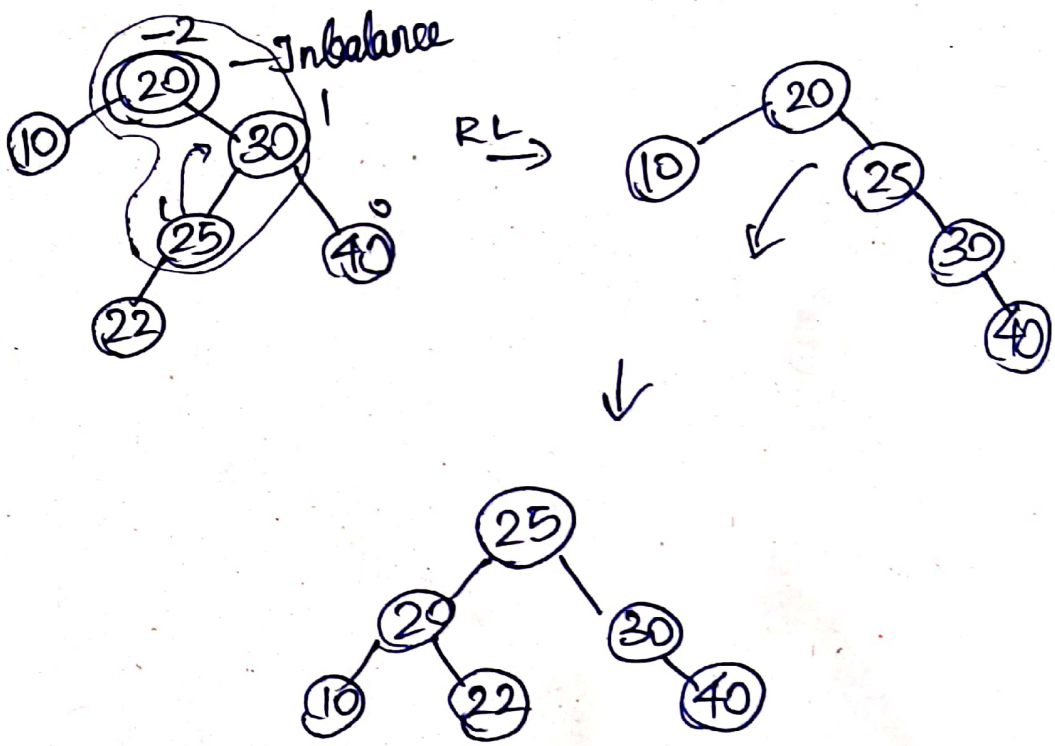


Scanned with OKEN Scanner

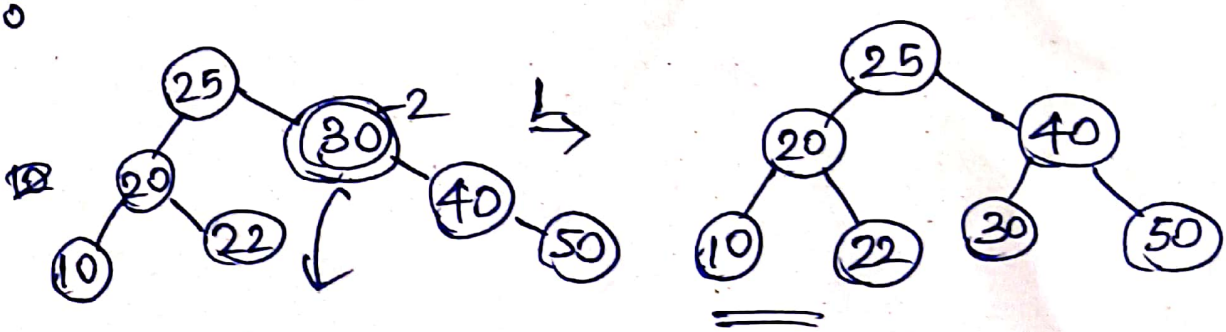
Insert 30



Insert 22



Insert 50



Scanned with OKEN Scanner